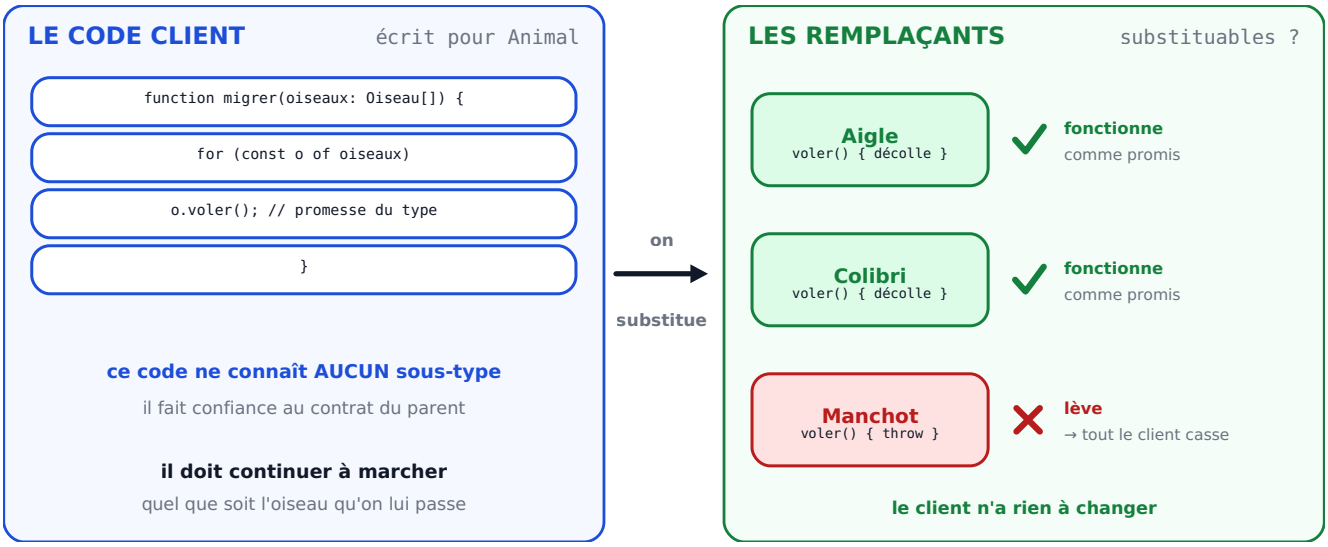
SOLID · LE L

Liskov /Substitution

Un sous-type doit pouvoir remplacer son parent

Tout le principe en schémas : les 4 clauses du contrat, les deux barres pré/post-condition, le test de substitution, et 10 cas réels découpés sous tes yeux.



1	Le contrat en 4 clauses	2
2	Les deux barres : précondition / postcondition	3
3	Le test de substitution (contract test)	4
4	Anatomie de la violation · du respect	5-6
5	10 cas réels, avant / après en images + code	7-11
6	Le reste de la boîte en un coup d'œil	12
7	Les pièges : hériter pour réutiliser	13
8	Arbre de décision · récap visuel	14-15

> Pour réviser : les pages 3 et 4 suffisent — les **deux barres** (entrée plus large, sortie plus garantie) et le **test de substitution**. Le reste n'est que déclinaison de ces deux idées.

« Si tu remplaces le parent par l'enfant, rien ne doit casser. »

Liskov Substitution Principle

Le sous-type hérite du contrat et de ses promesses

Hériter d'une classe, ce n'est pas hériter que des méthodes : c'est aussi hériter de ce qu'elles **garantissent**.

Le sous-type hérite du contrat… et de toutes ses promesses.

Préconditions · postconditions · invariants · exceptions

CLAUSE DU CONTRAT	LE PARENT PROMET	L'ENFANT PEUT	L'ENFANT NE PEUT PAS
Précondition	accepter [10 … 90]	✓ accepter [0 … 100] (plus large)	✗ n'accepter que [30 … 70]
Postcondition	renvoie un total ≥ 0	✓ garantir aussi la devise	✗ pouvoir renvoyer null
Invariant	largeur ≠ hauteur	– — (le préserver)	✗ setLargeur() change la hauteur
Exceptions	ne lève jamais d'erreur	– — (n'en ajoute pas)	✗ lever NotSupportedError

Enfant plus permissif en entrée, plus garantissant en sortie = substituable.

AVANT — l'enfant casse l'invariant

```
class Rectangle {
  constructor(public l: number, public h: number) {}
  setLargeur(l: number) { this.l = l }
  setHauteur(h: number) { this.h = h }
  aire() { return this.l * this.h }
}
class Carre extends Rectangle {
  setLargeur(l: number) { this.l = l; this.h = l }
}
```

APRÈS — deux types, un contrat

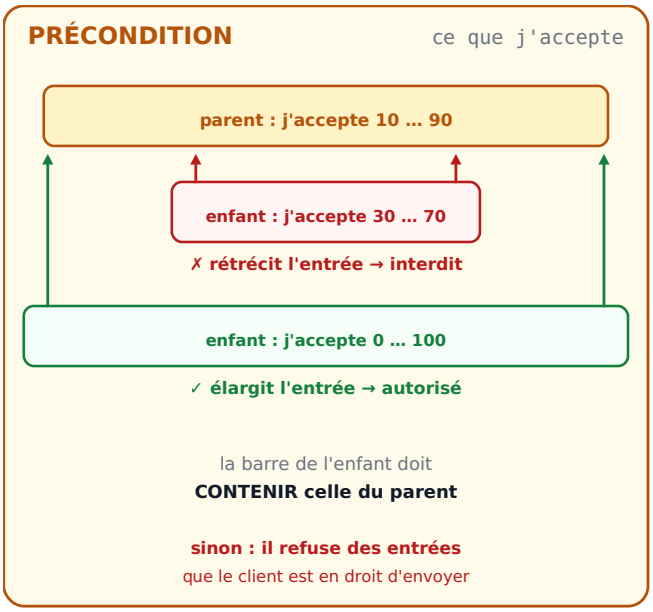
```
interface Forme { aire(): number }

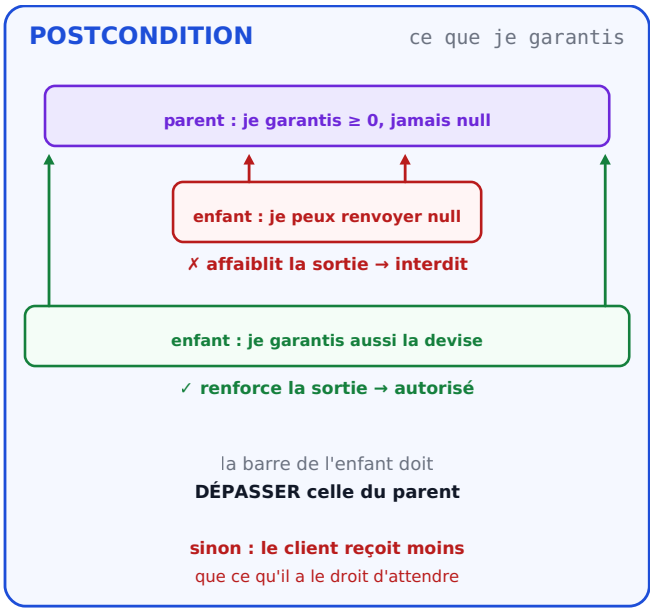
class Rectangle implements Forme {
  constructor(private l: number, private h: number) {}
  aire() { return this.l * this.h }
}
class Carre implements Forme {
  constructor(private cote: number) {}
  aire() { return this.cote ** 2 }
}
// plus d'héritage : plus de promesse implicite
```

RENDRE LE CONTRAT EXPLICITE	CE QUE TU ÉCRIS	CE QUE TU ÉVITES
Types et optionalité	`find(id): Client` — jamais `| null`	un enfant qui renvoie `null`
Exceptions	`@throws NotFound` ou `Result<T, E>`	une exception runtime inconnue du client
Invariants	`readonly` + tests de propriété	un `setLargeur()` qui change la hauteur
Effets de bord	« émet `facture.créée` » dans le contrat	un override qui n'émet plus rien

Entrée plus large, sortie plus garantie

Deux barres suffisent à vérifier 90 % des violations du LSP.





La formule à retenir : l'enfant peut être **plus permissif en entrée** et **plus généreux en sortie** — jamais l'inverse. C'est exactement la contravariance des paramètres et la covariance des retours, mais au niveau du **comportement**.

CE QUE FAIT L'ENFANT	VERDICT	POURQUOI
il accepte moins d'entrées que le parent	**violation**	le client envoie une entrée valide pour le parent et se fait refuser
il renvoie moins que ce que le parent garantit	**violation**	le client a écrit du code qui suppose la garantie
il lève une exception que le parent ne lève pas	**violation**	le client n'a aucun moyen de l'anticiper
il accepte plus large ou garantit plus	**conforme**	il tient toutes les promesses du parent, et davantage

AVANT — l'enfant ajoute une exception

```
// contrat : renvoie toujours un client
find(id: string): Client

// l'enfant :
find(id: string): Client {
  if (!this.ready)
    throw new Error("not ready")
  return this.inner.find(id)
}
```

APRÈS — l'échec est dans le contrat

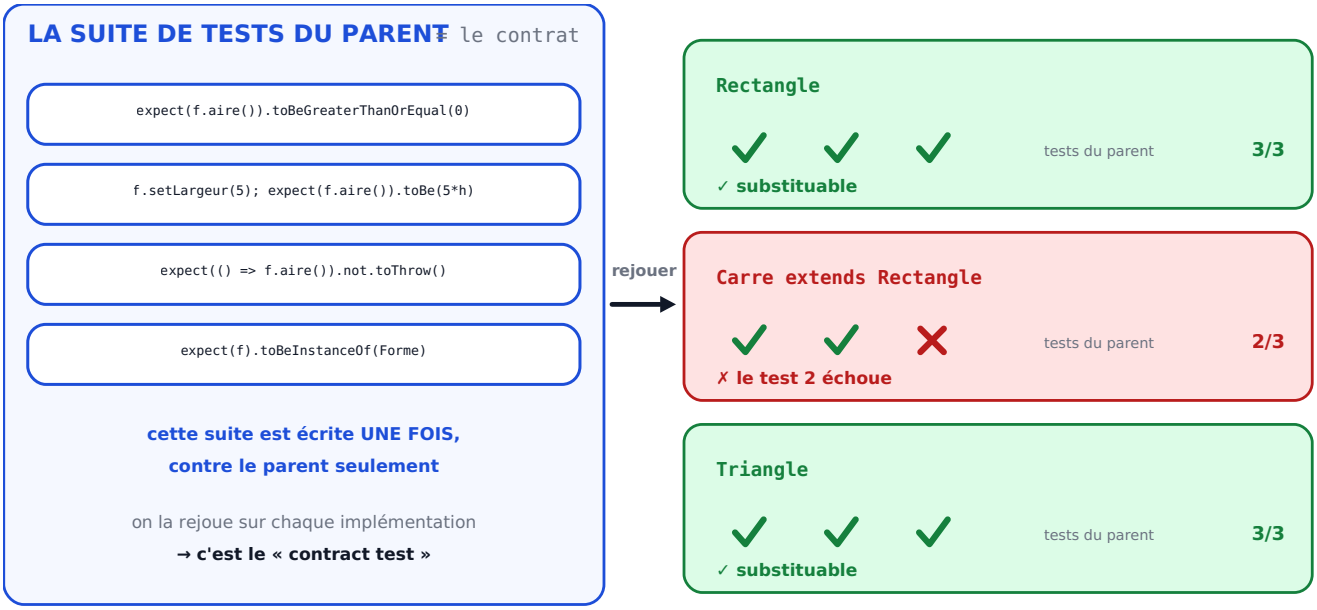
```
// le contrat dit ce qui peut arriver
find(id: string): Result<Client, "absent" | "indispo">

// chaque implémentation :
find(id): Result<Client, "absent" | "indispo"> {
  // même forme de retour pour tous
  // le client traite les 2 cas, une fois
}
```

3 · LE TEST

Le test de substitution

« Puis-je passer cet objet à tout le code écrit pour son parent ? » La réponse se **mesure** : une suite de tests partagée.



Un seul test rouge sur l'enfant = violation du LSP.

Peu importe que le code « marche » : le contrat, lui, ne tient plus.

A · Comment l'écrire

Une suite **écrite pour le parent**, exécutée sur chaque implémentation. Les cas viennent du contrat, pas du code de l'enfant.

B · Pourquoi ça marche

Le test est le seul endroit où le contrat **devient exécutable**. Ce qui n'est pas testé n'est pas garanti.

C · Le réflexe

Avant d'ajouter un héritage : écris le test du parent, puis lance-le sur l'enfant. S'il rougit, tu as ta réponse.

Le contract test, écrit une seule fois

```
// forme.contract.test.ts — la suite décrit le CONTRAT, pas une implémentation
describe.each([Rectangle, Carre, Triangle])("Forme : %p", (Forme) => {
  it("aire() est toujours ≥ 0", () => { expect(new Forme(...).aire()).toBeGreaterThanOrEqual(0) });
  it("setLargeur(5) puis setHauteur(4) donne une aire de 20", () => { … });
  it("ne lève jamais d'exception sur une entrée valide", () => { … });
});
```

Anatomie de la violation

Trois façons de casser le même contrat — et un seul symptôme côté client.

1 parent, 3 enfants qui ne tiennent pas parole

```
class Oiseau { voler() { … } couver() { … } }
```

Manchot — lève une exception

```
voler() { throw new Error(…); }
```

Autruche — override vide

```
couver() { /* vide */ }
```

Pingouin — retour affaibli

```
voler() { return null; }
```

trois façons de casser le même contrat

Ce que le client doit écrire

```
for (const o of oiseaux) {
if (o instanceof Manchot)
continue; // ← il sait
if (o instanceof Autruche)
o.voler(); // ← et devine
else o.voler();
}
```

✗ le client connaît les sous-types

✗ il devine ce que le type garantit

Le symptôme :

des instanceof partout dans le code métier.

A · Signal n°1

`throw new Error("not supported")` dans une méthode override.

B · Signal n°2

Un override **vide**, ou qui renvoie `null` / `0` / `[]` là où le parent garantit une valeur.

C · Signal n°3

Des `instanceof` ou `if (o.type === …)` dans le code client : il connaît les sous-types.

CE QUE LE CLIENT ÉCRIT	QUAND LE CONTRAT TIENT	QUAND IL EST VIOLÉ
`oiseau.voler()` sans condition	toujours correct	il doit filtrer `instanceof Manchot`
il suppose « jamais null »	il n'écrit aucun garde-fou	il ajoute `if (!x) return` partout
il suppose « ne lève pas »	il n'a pas de `try/catch`	il enveloppe tout dans un `try` défensif

Détecter une violation en 30 secondes

1 `grep -rn "not supported" src/`

un throw dans un override

→ on regarde ce qu'il trouve

2 `grep -rn "instanceof" src/`

le client connaît les sous-types

→ on regarde ce qu'il trouve

3 `contract test du parent`

un seul rouge = violation

→ on lance sur l'enfant

5 · LA CIBLE

Anatomie du respect

Chaque type ne promet que ce qu'il tient — et le client redevient simple.

Après : chaque type ne promet que ce qu'il tient

	précondition	postcondition	invariant	exceptions
Oiseau (base) `déplacer()`	✓	✓	✓	✓
Aigle : Volant `voler()`	✓	✓	✓	✓
Colibri : Volant `voler()`	✓	✓	✓	✓
Manchot : Oiseau `(rien à override)`	✓	✓	✓	✓

0 contrat cassé · 0 instanceof dans le client

Les 3 gestes de correction, dans cet ordre

1 Renommer / préciser

le parent promet-il vraiment ça ? souvent l'héritage était une erreur de vocabulaire.

2 Séparer le contrat

Volant · Remboursable · ClientEcriture : le sous-type ne prend que ce qu'il tient.

3 Composer

la sous-classe devient un décorateur ou une stratégie qui reçoit le parent.

Le critère de réussite : tu peux brancher une nouvelle implémentation **sans relire le client**, et la suite de tests du parent passe au vert sans une seule ligne modifiée.

A · Le client redevient bête

Plus de `instanceof`, plus de garde-fou défensif : il appelle, ça marche.

B · Les types se remplacent

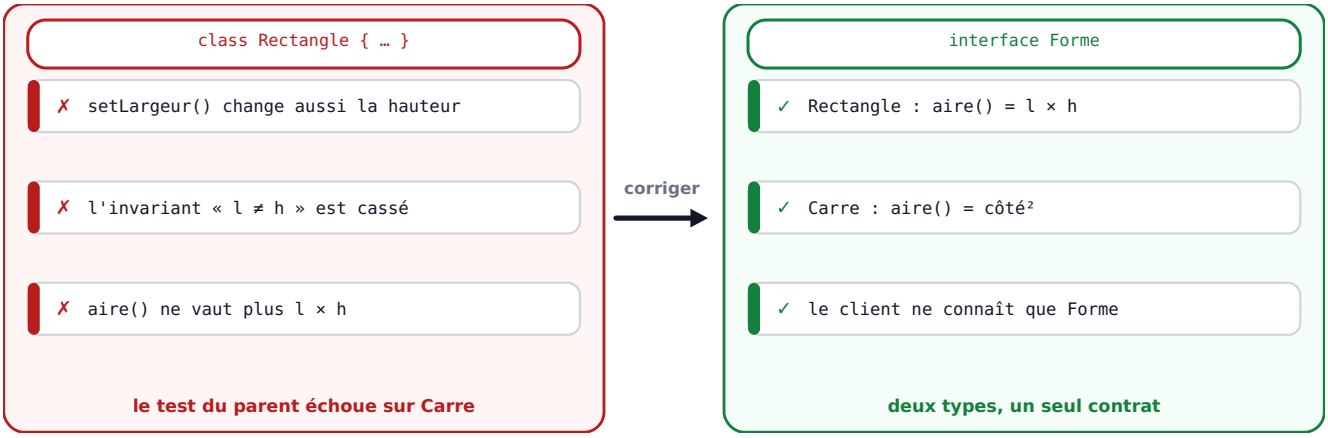
Une nouvelle implémentation se branche **sans relire le client** — c'est tout l'intérêt.

C · Les tests se partagent

La même suite tourne sur toutes les implémentations : une seule source de vérité sur le contrat.

1 Rectangle / Carré

l'exemple d'école : l'invariant du parent



AVANT

```
// rectangle.ts
class Rectangle {
  constructor(public l: number, public h: number) {}
  setLargeur(l: number) { this.l = l }
  setHauteur(h: number) { this.h = h }
  aire() { return this.l * this.h }
}
class Carre extends Rectangle {
  setLargeur(l: number) { this.l = l; this.h = l }
  setHauteur(h: number) { this.l = h; this.h = h }
}
```

APRÈS

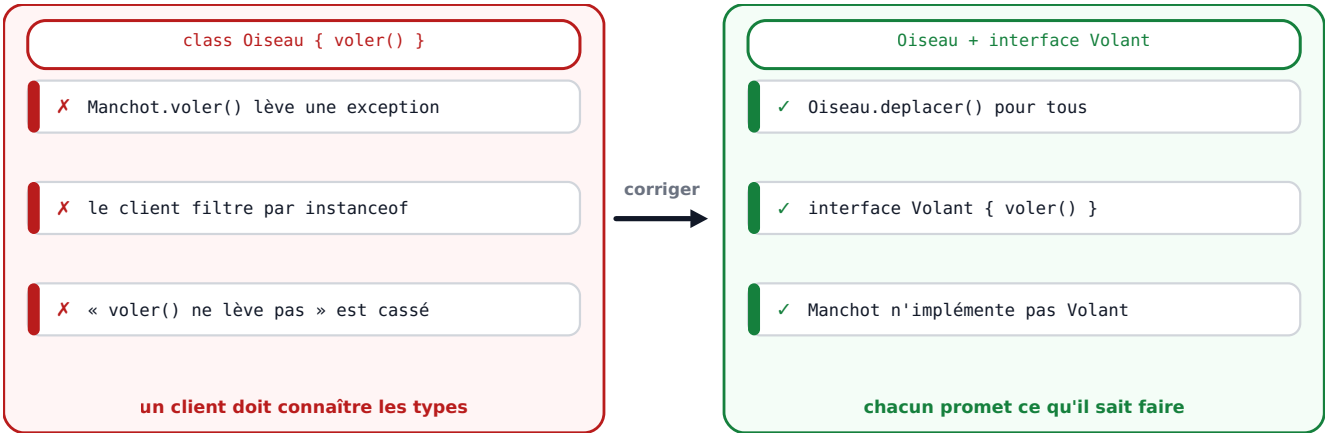
```
// forme.ts — pas d'héritage
interface Forme { aire(): number }

class Rectangle implements Forme {
  constructor(private l: number, private h: number) {}
  aire() { return this.l * this.h }
}
class Carre implements Forme {
  constructor(private cote: number) {}
  aire() { return this.cote ** 2 }
}
// client : function total(f: Forme[]) { … }
```

À retenir : le contrat implicite disait « largeur et hauteur **varient indépendamment** ». Aucune classe enfant ne peut tenir cette promesse si elle est un carré — donc le carré **n'est pas** un rectangle, en code.

2 Oiseau / Manchot

la méthode que le sous-type ne sait pas faire



AVANT

```
// oiseau.ts
class Oiseau { voler() { /* décolle */ } }

class Manchot extends Oiseau {
  voler() { throw new Error("je ne vole pas") }
}

function migrer(o: Oiseau[]) {
  for (const x of o) x.voler();  // casse
}
```

APRÈS

```
// oiseau.ts
class Oiseau { deplacer() { /* à pied, à tire d'aile */ } }
interface Volant { voler(): void }

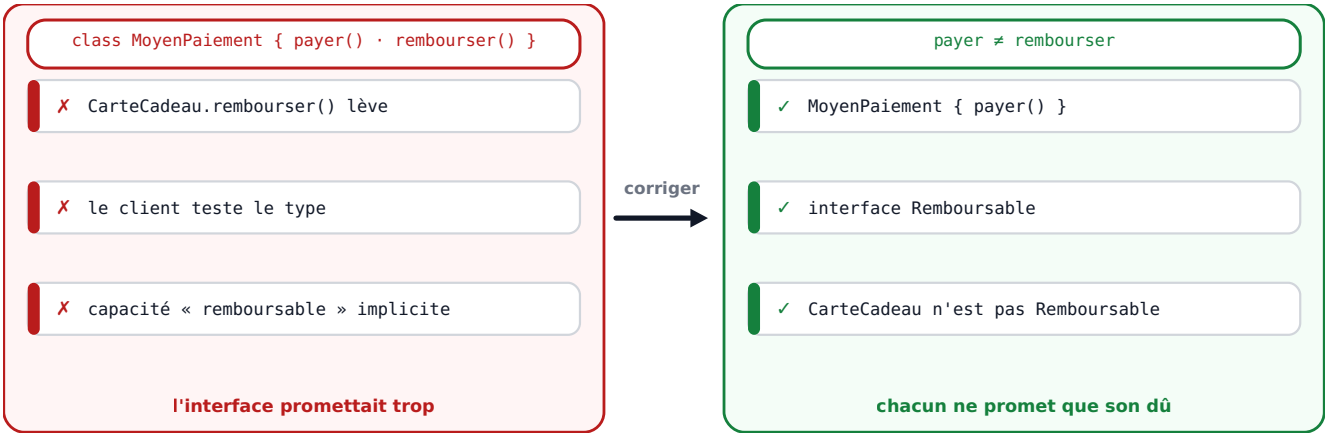
class Aigle extends Oiseau implements Volant {
  voler() { /* décolle */ }
}
class Manchot extends Oiseau { /* rien à override */ }

function migrer(v: Volant[]) {
  for (const x of v) x.voler()
}
```

À retenir : c'est aussi une leçon d'**ISP** : en séparant `Volant`, on empêche un oiseau incapable de voler d'hériter d'une promesse qu'il ne peut pas tenir.

3 Remboursement non supporté

une capacité que certains moyens n'ont pas



AVANT

```
// paiement.ts
class MoyenPaiement {
  payer(montant: number) { /* … */ }
  rembourser(id: string) { /* … */ }
}
class CarteCadeau extends MoyenPaiement {
  rembourser(id: string) {
    throw new Error("non remboursable")
  }
}
```

APRÈS

```
// paiement.ts
interface MoyenPaiement { payer(montant: number): void }
interface Remboursable { rembourser(id: string): void }

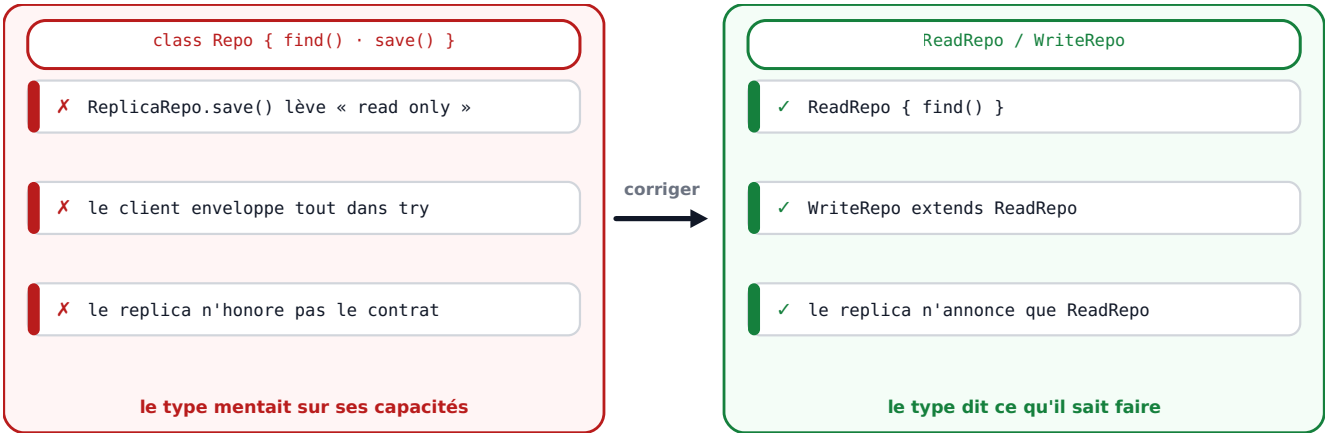
class Carte
  implements MoyenPaiement, Remboursable { … }
class CarteCadeau implements MoyenPaiement { … }

// le code qui rembourse exige Remboursable :
// function annuler(r: Remboursable) { … }
```

À retenir : un sous-type peut toujours **réduire** ce qu'il implémente ; il ne peut jamais **simuler** une capacité absente. La découpe se fait donc dans le parent, pas dans l'enfant.

4 Repository en lecture seule

le save() qui n'existe pas vraiment



AVANT

```
// repo.ts
class Repo {
  find(id: string) { /* SELECT */ }
  save(e: Entity) { /* UPDATE */ }
}
class ReplicaRepo extends Repo {
  save(e: Entity) { throw new Error("read only") }
}
```

APRÈS

```
// repo.ts
interface ReadRepo { find(id: string): Entity }

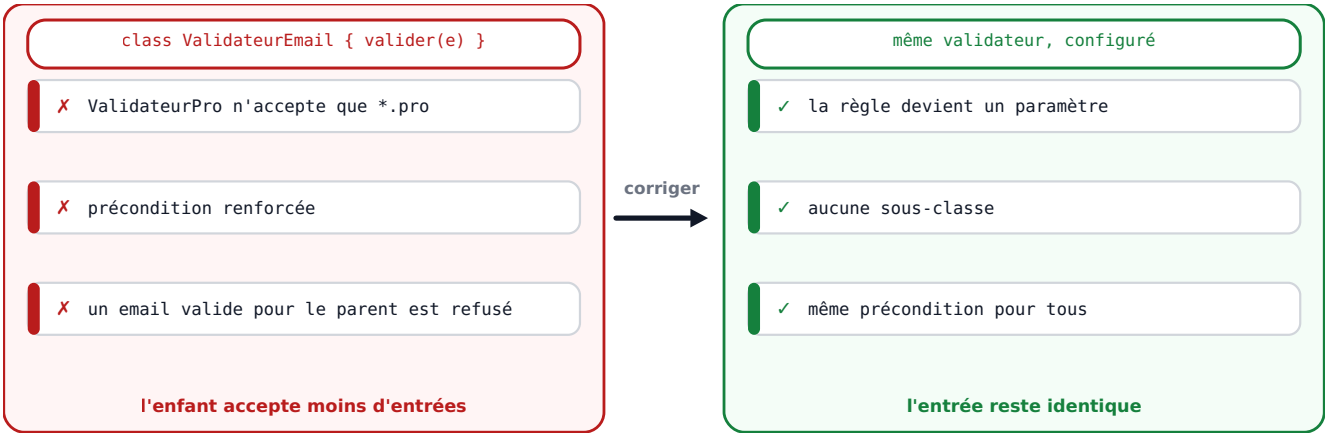
interface WriteRepo extends ReadRepo {
  save(e: Entity): void
}

// le code qui écrit demande WriteRepo
// le replica n'expose que ReadRepo
// → impossible de l'utiliser pour écrire
```

À retenir : ici la violation est **structurelle** : le type promettait save(). En séparant l'interface, le compilateur t'empêche d'appeler ce qui n'existe pas.

5 Précondition renforcée

l'enfant qui accepte moins que le parent



AVANT

```
// validation.ts
class ValidateurEmail {
  valider(e: string) { /* format */ }
}
class ValidateurPro extends ValidateurEmail {
  valider(e: string) {
    if (!e.endsWith(".pro"))
      throw new Error("domaine")
    super.valider(e)
  }
}
```

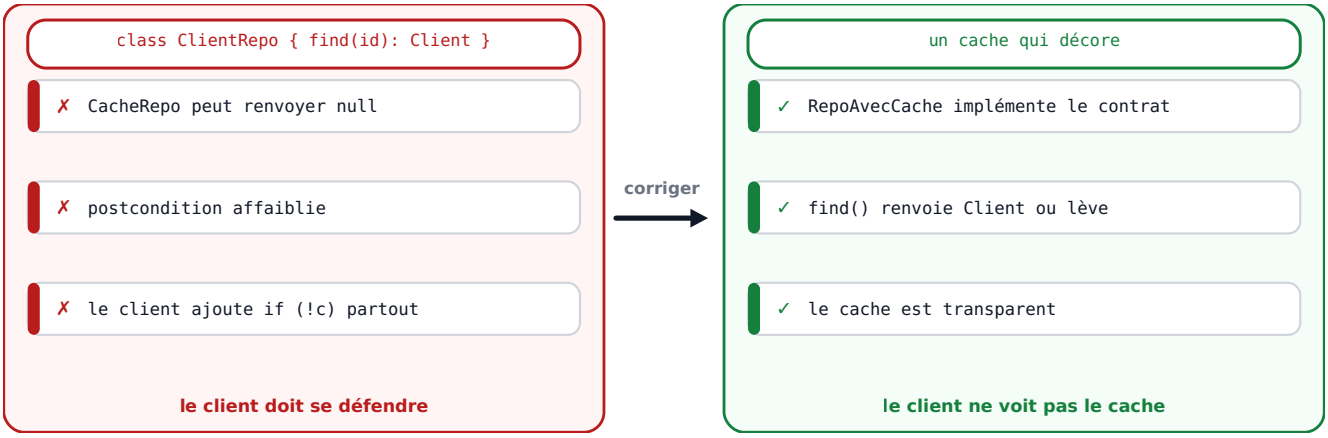
APRÈS

```
// validation.ts
class ValidateurEmail {
  constructor(private domaines?: string[]) {}
  valider(e: string) {
    if (!/@/.test(e)) throw new Error("format")   // même
    if (this.domaines &&          // configuré ?
        !this.domaines.some(d => e.endsWith(d)))
      throw new Error("domaine")
  }
}
// new ValidateurEmail([".pro"]) — pas de sous-classe
```

À retenir : une **variante de règle** n'est pas un sous-type. Paramétrer est plus sûr que spécialiser : le contrat d'entrée reste strictement le même.

6 Postcondition affaiblie

le cache qui renvoie null



AVANT

```
// repo.ts — contrat : renvoie TOUJOURS un client
class ClientRepo {
  find(id: string): Client { /* ou throw NotFound */ }
}
class CacheRepo extends ClientRepo {
  find(id: string): Client | null {
    return this.cache.get(id) ?? null  // affaiblie
  }
}
```

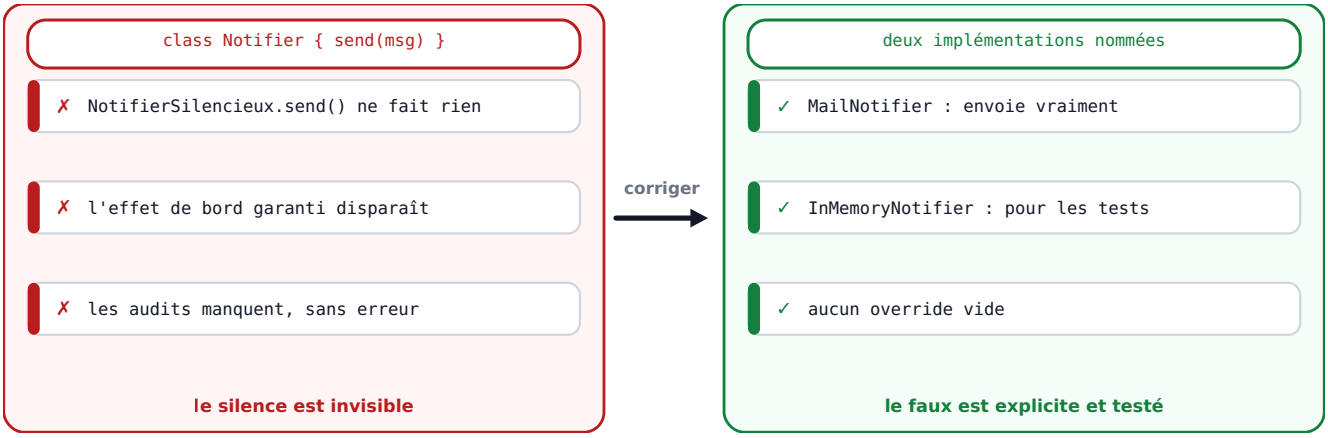
APRÈS

```
// repo-avec-cache.ts — composition
class RepoAvecCache implements ClientRepo {
  constructor(private inner: ClientRepo) {}
  find(id: string): Client {           // même contrat
    const hit = this.cache.get(id)
    if (hit) return hit
    const c = this.inner.find(id)      // lève NotFound
    this.cache.set(id, c)
    return c
  }
}
```

À retenir : le cache ne doit jamais **changer la garantie**. S'il peut rater, il lève comme le parent — sinon il ne doit pas s'appeler `ClientRepo`.

7 Override vide

le notifier qui n'envoie plus rien



AVANT

```
// notifier.ts
class Notifier {
  send(msg: string) { mail.send(msg) }
}
class NotifierSilencieux extends Notifier {
  send(msg: string) { /* rien */ }
}
// en prod : on branche le silencieux « pour aller vite »
```

APRÈS

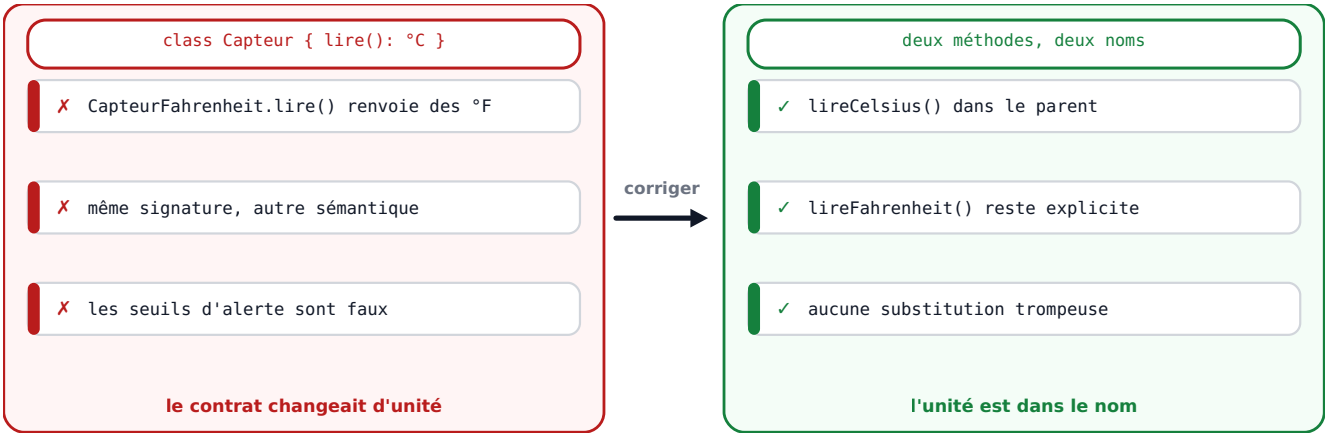
```
// notifier.ts
interface Notifier { send(msg: string): void }

class MailNotifier implements Notifier {
  send(msg: string) { mail.send(msg) }
}
class InMemoryNotifier implements Notifier {
  readonly envoyes: string[] = []
  send(msg: string) { this.envoyes.push(msg) }
}
// le test vérifie ce qui a été « envoyé »
```

À retenir : neutraliser un comportement par un override vide crée un **trou silencieux** dans le contrat. Un faux nommé et observable est toujours préférable.

8 Changement d'unité

la même méthode, un autre sens



AVANT

```
// capteur.ts — le contrat promet des °C
class Capteur {
  lire(): number { return this.sonde.celsius }
}
class CapteurFahrenheit extends Capteur {
  lire(): number { return this.sonde.fahrenheit }
}
// if (capteur.lire() > 30) alerter();  // faux
```

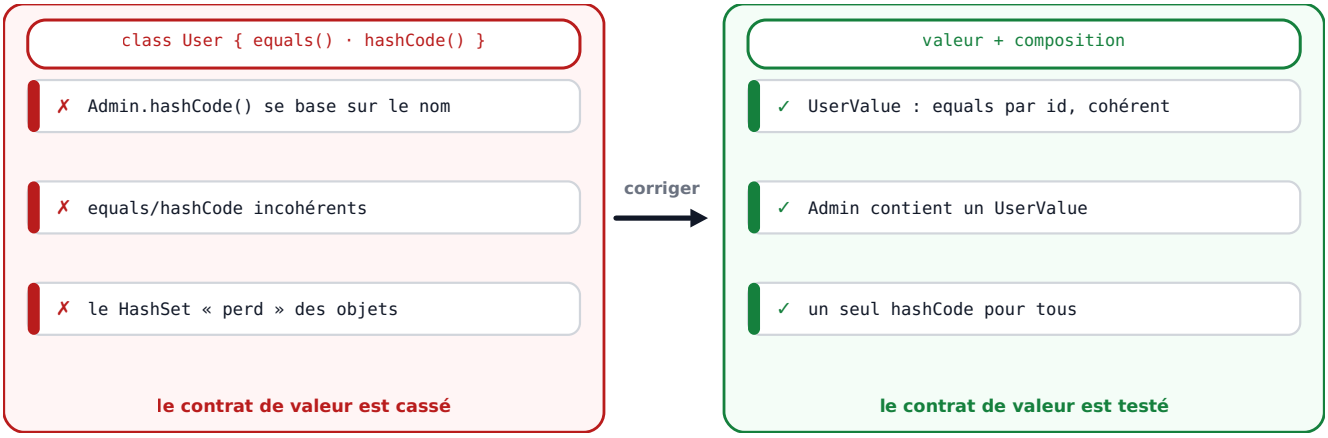
APRÈS

```
// capteur.ts
class Capteur {
  lireCelsius(): number { return this.sonde.celsius }
}
class CapteurFahrenheit {
  lireFahrenheit(): number { return this.sonde.f }
  lireCelsius(): number {
    return (this.sonde.f - 32) / 1.8   // converti
  }
}
// aucune méthode ne change de sens selon la classe
```

À retenir : une substitution qui **change l'unité, le format ou la devise** est une violation invisible par le compilateur — et dévastatrice à l'exécution. Le nom de la méthode doit porter l'unité.

9 equals / hashCode

un contrat non écrit mais très réel



AVANT

```
// user.ts
class User {
  equals(u: User) { return this.id === u.id }
  hashCode() { return hash(this.id) }
}
class Admin extends User {
  hashCode() { return this.nom.length }   // ≠ de equals
}
// set.add(admin); set.has(admin)  → false
```

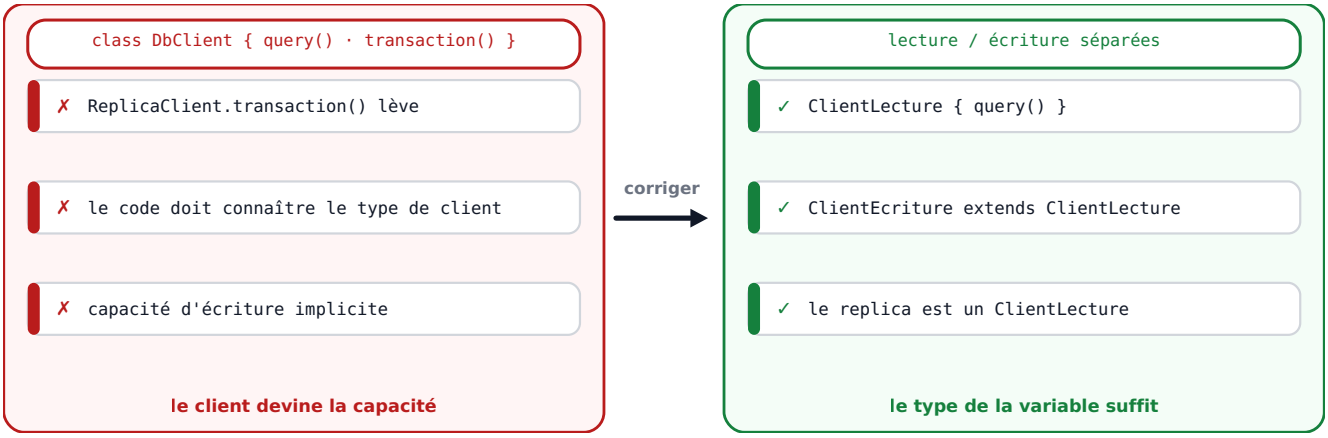
APRÈS

```
// user-value.ts — value object, jamais étendu
class UserValue {
  equals(u: UserValue) { return this.id === u.id }
  hashCode() { return hash(this.id) }
}
// admin.ts
class Admin {
  constructor(public readonly user: UserValue) {}
  // aucune override : on compose
}
// test partagé : equals ⇒ même hashCode
```

À retenir : equals/hashCode forment un **contrat** (réflexivité, symétrie, cohérence). Un enfant qui n'en redéfinit qu'un seul le brise — silencieusement.

10 Client DB en lecture seule

la transaction qui n'existe que sur le primaire



AVANT

```
// db.ts
class DbClient {
  query(sql: string) { /* … */ }
  transaction<T>(fn: () => T) { /* … */ }
}
class ReplicaClient extends DbClient {
  transaction<T>(fn: () => T) {
    throw new Error("lecture seule")
  }
}
```

APRÈS

```
// db.ts
interface ClientLecture { query(sql: string): Rows }

interface ClientEcriture extends ClientLecture {
  transaction<T>(fn: () => T): T
}

// le replica est déclaré ClientLecture
// le primaire est déclaré ClientEcriture
// → écriture refusée sur le replica
```

À retenir : quand un sous-type ne peut pas honorer une méthode, la vraie réponse est presque toujours **dans la découpe du parent** — pas dans l'enfant.

6 · TOUR D'HORIZON

Le reste de la boîte, en un coup d'œil

Six violations que tu as forcément croisées : à gauche le symptôme, à droite la découpe.

Stack extends ArrayList
hériter juste pour réutiliser

```
push(x) { this.add(x) }
pop() { this.remove(0) }
add(i, x) { throw }
```

→

```
class Stack {
items: T[] = []
push(x) { items.push }
```

héritage non tenu
composer, pas étendre

Comparator incohérent
0 pour des valeurs ≠

```
compare(a, b) {
return a.id - b.id
}
```

→

```
tri stable garanti
même ordre partout
```

contrat de tri cassé
le tri devient aléatoire

DiscountPolicy négatif
la remise peut dépasser le total

```
apply(t) {
return t - 100
}
```

→

```
apply(t): Montant ≥ 0
borné par le type
```

postcondition violée
un total négatif en prod

Animal.sound()
le poisson n'a pas de son

```
class Poisson extends Animal
sound() { throw }
```

→

```
interface Bruyant
sound() hors du parent
```

capacité inventée
séparer le contrat

FileStorage.write()
un stockage d'archives immuable

```
class Archive
write() { throw }
```

→

```
ReadOnlyStorage
sans write()
```

capacité en trop
découper l'interface

Shape.area() = 0
l'échec silencieux

```
class FormeLibre
area() { return 0 }
```

→

```
throw / pas de méthode
l'erreur est visible
```

faux résultat
0 n'est pas « inconnu »

A · Le réflexe

Un `throw` dans un override est un **aveu** : la méthode n'a rien à faire dans le parent.

B · La question

« Est-ce que mon enfant peut **tout** ce que le parent promet ? » Si non, ce n'est pas un sous-type.

C · Le piège

Un enfant qui renvoie `0` ou `null` est plus dangereux qu'un `throw` : il échoue **en silence**.

Hériter pour réutiliser du code

C'est la cause n°1 des violations : on veut **le code** du parent, mais on hérite aussi de ses **promesses**.

HÉRITER POUR RÉUTILISER — le piège n°1

```
class Stack<T> extends ArrayList<T> {
push(x: T) { this.add(x) }
pop(): T { return this.remove(0) }
}
// le client peut appeler stack.add(0, x)
```

- ✗ on hérite d'une API qu'on ne tient pas
- ✗ add(i, x) va lever chez nous
- ✗ le client peut appeler 40 méthodes

on hérite d'un contrat qu'on ne tient pas

COMPOSER — le bon geste

```
class Stack<T> {
private items: T[] = []
push(x: T) { this.items.push(x) }
pop(): T { return this.items.pop()! }
}
```

- ✓ l'API publique = ce qu'on promet
- ✓ aucun contrat hérité, aucune surprise
- ✓ le test unitaire suffit

on réutilise par délégation, pas par héritage

1 · Le contrat implicite

Un contrat non écrit n'est pas **inexistant** : il est inventé par chaque lecteur. Écris-le (types, exceptions, `Result`).

2 · LSP ≠ héritage

Le principe parle de **substituabilité** : interfaces et duck typing y sont soumis aussi. Une implémentation d'interface peut violer le LSP.

3 · Le faux objet de test

Un mock qui renvoie autre chose que le contrat rend les tests verts et la prod rouge. Le faux doit être un **vrai** sous-type.

Quand l'héritage est légitime — et quand il devient un piège

LÉGITIME

- ✓ le type est vraiment un cas particulier
- ✓ l'enfant ne refuse aucun comportement
- ✓ le contract test du parent passe

PIÈGE

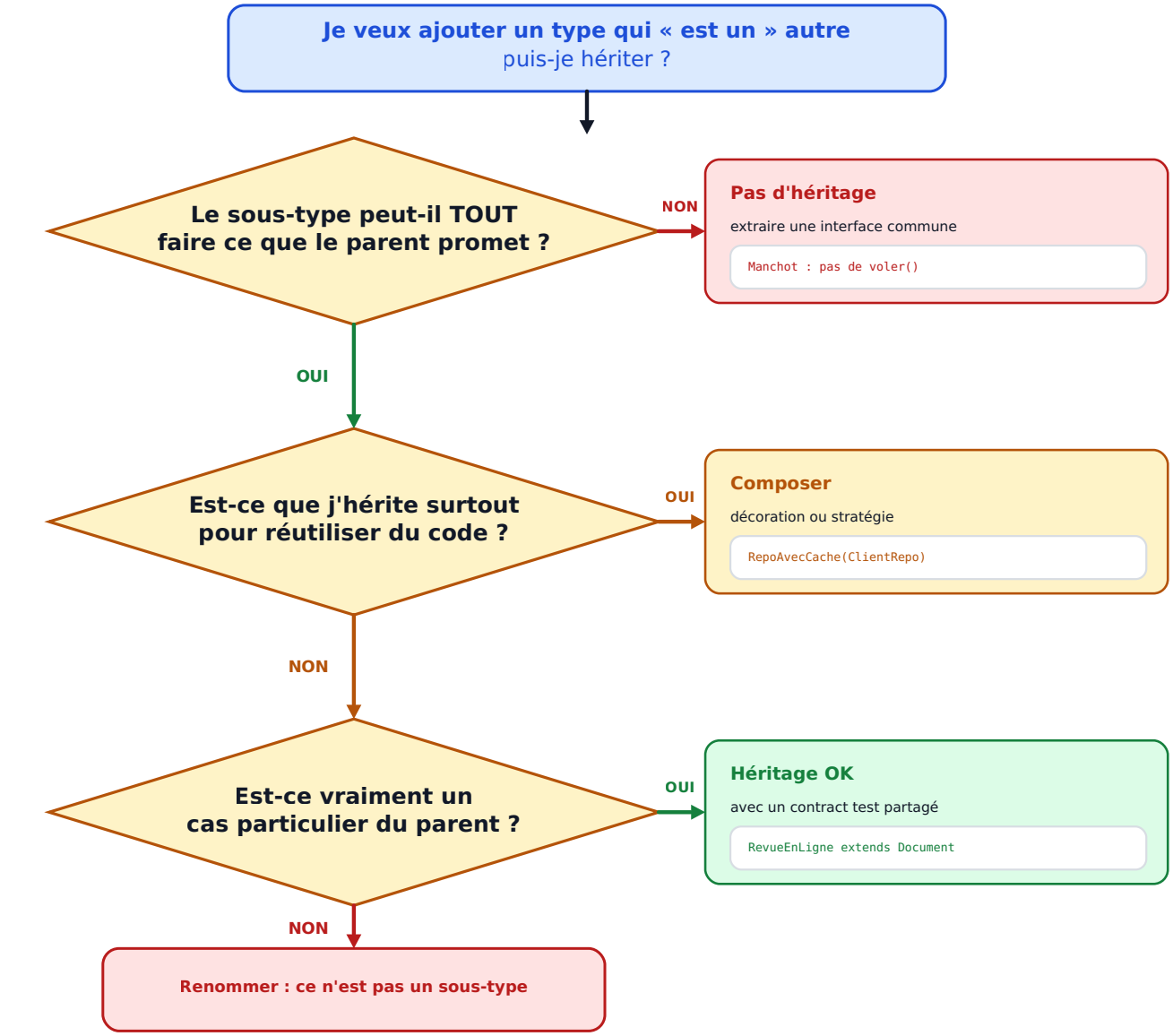
- ✗ on veut surtout éviter la duplication
- ✗ on veut un type « presque pareil »
- ✗ le parent a des méthodes concrètes non génériques

LE SIGNAL DANS LE CODE	CE QU'IL RÉVÈLE	LE GESTE
`throw new Error("not supported")`	l'enfant refuse un comportement annoncé	séparer l'interface (ISP)
un override **vide**	l'enfant annule un effet promis	composer / renommer
`instanceof` chez le client	le contrat du parent est faux	déplacer la méthode
une unité ou un format différent	même signature, autre sens	renommer la méthode
`equals` sans `hashCode`	le contrat de valeur est cassé	value object figé

8 · DÉCIDER

Dois-je hériter ?

Trois questions, dans cet ordre. La première élimine 90 % des cas.



Dans le doute : écris le test du parent et lance-le sur l'enfant.

S'il rougit, tu n'as pas un sous-type — tu as un type voisin.

Carré → Forme

le carré n'étend pas le rectangle

Replica → ReadRepo

le replica n'annonce que la lecture

Cache → composition

le cache enveloppe, il n'hérite pas

Le raccourci : hériter, c'est **promettre**. On hérite pour le code, mais on hérite aussi des comportements — et c'est là que tout casse. Chaque question de l'arbre te fait vérifier une promesse, pas une commodité.

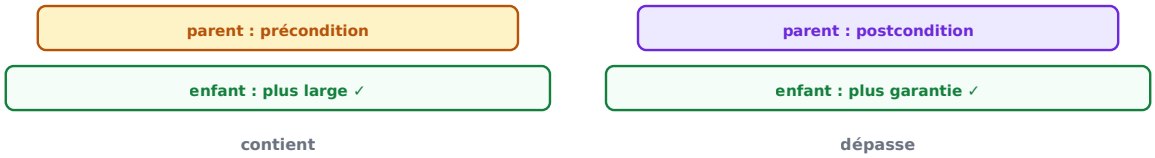
9 · RÉVISER

Les 4 clauses, et le réflexe visuel

Si tu retiens une page, c'est celle-ci.



Le réflexe visuel : la barre de l'enfant



En une phrase ?

Un sous-type doit pouvoir remplacer son parent sans rien casser.

Les 4 règles ?

Préconditions non renforcées · postconditions non affaiblies · invariants préservés · pas d'exception en plus.

Le test ?

La suite de tests du parent reste verte sur chaque implémentation.

Les signaux ?

`throw "not supported"` · override vide · `instanceof` chez le client · `null` inattendu · unité changée.

Les 3 corrections ?

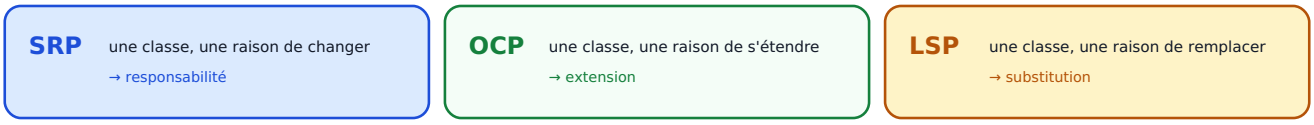
Renommer/préciser · séparer le contrat (ISP) · composer.

Le lien avec l'ISP ?

LSP dit « tiens la promesse » ; ISP dit « ne promets pas trop ».

CHECKLIST AVANT DE VALIDER UN SOUS-TYPE	SI NON →
Toutes les méthodes du parent ont-elles un sens ici ?	c'est un type voisin, pas un sous-type
Aucune méthode ne lève « non supporté » ?	séparer l'interface (ISP)
Les préconditions sont-elles **au moins aussi larges** ?	violation : l'enfant refuse une entrée valide
Les postconditions sont-elles **au moins aussi fortes** ?	violation : le client reçoit moins que prévu
La suite de tests du parent passe-t-elle **sans modification** ?	le contrat n'est pas tenu

Les trois principes côte à côte



Et si tu oublies tout le reste : avant d'écrire `extends`, demande-toi ce que tu promets. Si tu ne peux pas tenir **toutes** les promesses du parent, tu ne veux pas hériter — tu veux **composer**.